operations are performed on a device file, the appropriate functions from our driver's code get called. Let us perform the *read* operation and check the output of the *dmesg* command:

```
[root]# cat /dev/null_driver

[root]# dmesg
Calling: null_open
Calling: null_read
Calling: null_release
```

To make things simple I have used *printk()* statements in every function. If we remove these statements, then */dev/null_driver* will behave exactly the same as the */dev/null* pseudo device. Our code is working as expected. Let us understand the details of our character driver.

First, take a look at the driver's function. Given below are the prototypes of a few functions from the *file_operations* structure:

```
int   (*open)(struct inode *i, struct file *f);
int   (*release)(struct inode *i, struct file *f);
ssize_t (*read)(struct file *f, char __user *buf, size_t len,
loff_t *off);
ssize_t (*write)(struct file *f, const char __user buf*,
size_t len, loff_t *off);
```

The prototype of the *open()* and *release()* functions is exactly same. These functions accept two parameters—the first is the pointer to the *inode* structure. All file-related information such as size, owner, access permissions of the file, file creation timestamps, number of hard-links, etc, is represented by the *inode* structure. And each open file is represented internally by the *file* structure. The *open()* function is responsible for opening the device and allocation of required resources. The *release()* function does exactly the reverse job, which closes the device and deallocates the resources.

As the name suggests, the *read()* function reads data from the device and sends it to the application. The first parameter of this function is the pointer to the file structure. The second parameter is the user-space buffer. The third parameter is the size, which implies the number of bytes to be transferred to the user space buffer. And, finally, the fourth parameter is the file offset which updates the current file position. Whenever the *read()* operation is performed on a device file, the driver should copy len bytes of data from the device to the user-space buffer *buf* and update the file offset *off* accordingly. This function returns the number of bytes read successfully. Our null driver doesn't read anything; that is why the return value is always zero, i.e., EOF.

The driver's *write()* function accepts the data from the user-space application. The first parameter of this function is the pointer to the file structure. The second parameter is the user-space buffer, which holds the data received from the application. The third parameter is len which is the size of the data. The fourth parameter is the file offset. Whenever the *write()* operation is performed on a device file, the driver should transfer len bytes of data to the device and update the file offset *off* accordingly. Our *null* driver accepts input of any length; hence, return value is always *len*, i.e., all bytes are written successfully.

In the next step we have initialised the *file_operations* structure with the appropriate driver's function. In *initialisation* function we have done a registration related job, and we are deregistering the character device in *cleanup* function.

Implementation of the full pseudo driver

Let us implement one more pseudo device, namely, *full*. Any write operation on this device fails and gives the 'ENOSPC' error. This can be used to test how a program handles disk-full errors. Given below is the complete working code of the full driver:

```
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/fs.h>
#include <linux/kdev_t.h>

static int major;
static char *name = "full_driver";

static int full_open(struct inode *i, struct file *f)
{
     return 0;
}

static int full_release(struct inode *i, struct file *f)
{
     return 0;
}

static ssize_t full_read(struct file *f, char __user *buf,
size_t len, loff_t *off)
{
     return 0;
}

static ssize_t full_write(struct file *f, const char __user
*buf, size_t len, loff_t *off)
{
     return -ENOSPC;
}

static struct file_operations full_ops =
{
     .owner    = THIS_MODULE,
     .open     = full_open,
     .release  = full_release,
     .read     = full_read,
     .write    = full_write
};
```

To be continued on page.... 55